DARSHAN PRAJAPATI
MCA-1
ROLL NO: 24
SUBMISSION DATE: 15/09/2025

 # 1) Display the difference in dates

```
from datetime import datetime

# date1='2025-08-20'
# date2='2025-08-30'

date1=input('Enter first date ( YYYY-MM-DD): ')
date2=input('Enter second date ( YYYY-MM-DD): ')

d1=datetime.strptime(date1,"%Y-%m-%d")
d2=datetime.strptime(date2,"%Y-%m-%d")

difference=abs(d2 - d1)

print('Difference of dates in days: ',difference.days, ' days')

# output

# Enter first date ( YYYY-MM-DD): 2025-10-01
# Enter second date ( YYYY-MM-DD): 2025-11-01
# Difference of dates in days:  31  days
```

 # 2) Display time since epoch in hours and minutes

```
import time

epoch =time.time()

hours=int(epoch//3600)
minutes=int((epoch%3600)//60)
```

```
print("Time since epoch")
print(f"Hours: {hours} Minutes: {minutes}")

# output
# Time since epoch
# Hours: 488225 Minutes: 15
```

 # 3) Display your age in years, months and days

```
from datetime import datetime

# bday='2003-07-03'
bday=input('Enter birth date in formate (YYYY-MM-DD):')
today=datetime.now()

bday_formated=datetime.strptime(bday,'%Y-%m-%d')

diff_day=(today-bday_formated).days

years = diff_day // 365
remaining_days = diff_day % 365
months = remaining_days // 30
days = remaining_days % 30

print(f'Years:{years} months:{months} days:{days}')

# output
# Enter birth date in formate (YYYY-MM-DD):2003-07-20
# Years:22 months:1 days:29
```

 # 4) Display trigonometric table of sin, cos and tan

```
import math
angles=[0,30,45,60,90]

print('Angle \t\tsin \t\t cos \t\t tan ')
for angle in angles :
 rad=math.radians(angle)
```

```
 sin=round( math.sin(rad),4)
 cos= round(math.cos(rad),4)
 if(angle==90):tan='undefiend'
 else :
  tan= round(math.tan(rad),4)
 print(f'{angle} \t\t {sin} \t\t {cos} \t\t {tan}')

 # output
#  Angle          sin            cos            tan
# 0              0.0            1.0            0.0
# 30             0.5            0.866          0.5774
# 45             0.7071         0.7071         1.0
# 60             0.866          0.5            1.7321
# 90             1.0            0.0            undefiend
```

 # 5) Generate 10 random numbers

```
import random
print('Printing 10 random numbers')
for _ in range(10):
 num= random.randint(1,100)
 print(num)

# output

# Printing 10 random numbers
# 84
# 67
# 48
# 75
# 35
# 37
# 10
# 46
# 99
# 92
```

 # 6) Authentication Ask username, password and compare

```
userName=input("Enter your username: ")
password=input("Enter your Password: ")

correct_userName='user123'
correct_password='pass123'

if userName == correct_userName and password == correct_password:
    print("Authentication successful!")
else:
    print("Authentication failed!")

# output
# Enter your username: user123
# Enter your Password: pass123
# Authentication successful!

# or

# Enter your username: user112
# Enter your Password: pass123
# Authentication failed!
```

 # 7) Authentication Ask username, password and compare with encryption

```
from cryptography.fernet import Fernet
key=Fernet.generate_key()
cipher=Fernet(key)

correct_username='user123'
correct_password='pass123'.encode()
encrypted_password=cipher.encrypt(correct_password)

username=input("Enter your username: ")
password=input("Enter your password: ").encode()
```

```
if(username==correct_username and password==cipher.decrypt(encrypted_password)):
    print('Authentication Successful!')
else:
    print('Authentication Failed!')

# output
# Enter your username: user123
# Enter your password: pass123
# Authentication Successful!
```

 # 8) Authentication Ask username, password and compare with hashing

```
import hashlib

# correct username and password [ hashed ]
correct_username='user123'
correct_password=hashlib.sha256('pass123'.encode()).hexdigest()

# user inputs
username=input("Enter Username: ")
password=input("Enter Password: ")

# hash the input password
password_hash=hashlib.shake_256(password.encode()).hexdigest()

if(username==correct_username and password_hash==correct_password):
    print('Authentication Successful!')
else:
    print('Authentication Failed!')

# output
# Enter your username: user123
# Enter your password: pass123
# Authentication Successful!
```

 # 9) Convert string Hello$World into Base64

```
import base64

text='Hello$world'

# convert text into bytes
text_bytes=text.encode('utf-8')
# convert bytes into base64 bytes
base64_bytes=base64.b64encode(text_bytes)

# convert base64 bytes back to string
base64_string=base64_bytes.decode('utf-8')

print('Original string: ',text)
print('Base64 encoded string: ',base64_string)

# output
# Original string:  Hello$world
# Base64 encoded string:  SGVsbG8kd29ybGQ=
```

 # 10) Code for String Manipulation

```
# Exercise 1A:
#  Create a string made of the first, middle and last character
str1='james'
#first character
res=str1[0]
# mid character
mid=int(len(str1)/2)
res=res+str1[mid]
# last character
res=res+str1[-1]
# print(res)

# output
```

```
# jms

# Exercise 1B:
# Create a string made of the middle three characters
str2='JaSonAy'
mid=int(len(str2)/2)
res=str2[mid-1]+str2[mid]+str2[mid+1]
# print(res)

# Exercise 2:
# Append new string in the middle of a given string
s1 = "Ault"
s2 = "Kelly"

mid=int(len(s1)/2)
x=s1[:mid]
x+=s2
x+=s2[mid:]
# print(x)
# output
# AuKellylt

# Exercise 3:
#  Create a new string made of the first, middle, and last characters of each input string
s1 = "America"
s2 = "Japan"
res=s1[0]+s2[0]
mid1=int(len(s1)/2)
mid2=int(len(s2)/2)
res+=s1[mid1]+s2[mid2]
res+=s1[-1]+s2[-1]
# print(res)

# output
# AJrpan
```

```
# Exercise 4:
# Arrange string characters such that lowercase letters should come first
str1 = 'PyNaTive'
# print('Original String:', str1)
lowerlist=[]
upperlist=[]

for c in str1:
    if c.islower():
        lowerlist.append(c)
    else:
        upperlist.append(c)

sorted_list=''.join(lowerlist + upperlist)
# print("Result:",sorted_list)

# output
# Original String: PyNaTive
# Result: yaivePNT

# Exercise 5:
# Count all letters, digits, and special symbols from a given string

# str1 = "P@#yn26at^&i5ve"
def checkLettersDigitsSymbols(str1):
    char=0
    digits=0
    symbol=0
    for c in str1:
        if c.isalpha():
         char+=1
        elif c.isdigit():
            digits+=1
        else:
            symbol+=1
```

```
    print(f"Total counts {char} of chars,{digits} digits, and {symbol} symbols")
# checkLettersDigitsSymbols(str1)


# output
# Original String: PyNaTive
# Total counts 8 of chars,3 digits, and 4 symbols


# Exercise 6:
# Create a mixed String using the following rules
# Given two strings, s1 and s2.
# Write a program to create a new string s3 made of the first char of s1, then the last char of s2, Next, the second char of s1 and second last char of s2, and so on. Any leftover chars go at the end of the result.


s1 = "Abc"
s2 = "Xyz"
# Expected: 'AzbycX'


# get string length
s1_length = len(s1)
s2_length = len(s2)
result=""


# find length which ever is bigger
length=s1_length if s1_length>s2_length else s2_length


# reverse s2
s2=s2[: : -1]
for i in range(length):
    if i<s1_length:
        result+=s1[i]
    if i<s2_length:
        result+=s2[i]


# print(result)


# output
```

AzbycX

Exercise 7:

String characters balance Test

Write a program to check if two strings are balanced. For example, strings s1 and s2 are balanced if all the characters in the s1 are present in s2. The character's position doesn't matter

```
def checkCharactersBalanceTest(s1,s2):
   flag=True
   for i in s1:
     if i in s2:
      continue
     else:
      flag=False
   return flag

s1 = "Yn"
s2 = "PYnative"
# flag = checkCharactersBalanceTest(s1, s2)
# print("s1 and s2 are balanced:", flag)

s1 = "Ynf"
s2 = "PYnative"
# flag = checkCharactersBalanceTest(s1, s2)
# print("s1 and s2 are balanced:", flag)

# output
# s1 and s2 are balanced: True
# s1 and s2 are balanced: False
```

Exercise 8:

Find all occurrences of a substring in a given string by ignoring the case

Write a program to find all occurrences of "USA" in a given string ignoring the case

```
str1 = "Welcome to USA. usa awesome, isn't it?"
substring='USA'
```

```
# convert string into lowercase
temp_str=str1.lower()
# counts the occurances of substring
counts=temp_str.count(substring.lower())
# print(f"The {substring} count is: {counts}")

# output
# The USA count is: 2

# Exercise 9:
# Calculate the sum and average of the digits present in a string
# Given a string s1, write a program to return the sum and average of the digits that appear in the string, ignoring all other characters

str1 = "PYnative29@#8496"
sum=0
count=0
for i in str1:
  if i.isdigit():
    sum+=int(i)
    count+=1
# print("Sum :",sum, "Average: ", sum/count)

# Exercise 10:
#  Write a program to count occurrences of all characters within a string
str1 = "Apple"
char_dict=dict()
for char in str1:
  count= str1.count(char)
  char_dict[char]=count

# print(char_dict)

# output
# {'A': 1, 'p': 2, 'l': 1, 'e': 1}
```

```
# Exercise 11:
# Reverse a given string
# str1 = "PYnative"

rev=str1[::-1]
#or
rev2="".join(reversed(str1))
# print("Original String is:", str1)

str1 = ''.join(reversed(str1))
# print("Reversed String is:", rev)

# output
# Original String is: PYnative
# Reversed String is: evitanYP

# Exercise 12:
# Find the last position of a given substring
# Write a program to find the last position of a substring “Emma” in a given string.

str1 = "Emma is a data scientist who knows Python. Emma works at google."
# print("Original String is:", str1)

index = str1.rfind("Emma")
# print("Last occurrence of Emma starts at index:", index)

# output
# Original String is: Emma is a data scientist who knows Python. Emma works at google.
# Last occurrence of Emma starts at index: 43

# Exercise 13:
# Split a string on hyphens
# Write a program to split a given string on hyphens and display each substring.

str1 ="Emma-is-a-data-scientist"
new_str=str1.split('-')
```

```
# for ch in new_str:
#    print(ch)

# output
# Emma
# is
# a
# data
# scientist

# Exercise 14:
# Remove empty strings from a list of strings

str_list = ["Emma", "Jon", "", "Kelly", None, "Eric", ""]
new_str=[]
# using filter
new_str2=list(filter(None,str_list))

# using if condition
for s in str_list:
  if s :
    new_str.append(s)
# print("Empty string:",str_list)
# print("Non empty string:",new_str)
# print("Non empty string:",new_str2)

# output:
# Empty string: ['Emma', 'Jon', '', 'Kelly', None, 'Eric', '']
# Non empty string: ['Emma', 'Jon', 'Kelly', 'Eric']
# Non empty string: ['Emma', 'Jon', 'Kelly', 'Eric']

# Exercise 15:
# Remove special symbols / punctuation from a string
# Given:
str1 = "/*Jon is @developer & musician"
```

```
# Expected Output: "Jon is developer musician"
# print("Original string is: ", str1)
import re
# replace special symbols with ''
res=re.sub(r'[^\w\s]', '', str1)
# print("New string is: ",res)

# output
# Original string is:  /*Jon is @developer & musician
# New string is:  Jon is developer  musician

# Exercise 16:
# Removal all characters from a string except integers
# Given:
str1 = 'I am 25 years and 10 months old'
# Expected Output: 2510

res=" "
for ch in str1:
  if ch.isdigit():
    res+=''.join(ch)
# print(res)

# output:
# 2510

# Exercise 17:
# Find words with both alphabets and numbers
# Write a program to find words with both alphabets and numbers from an input string.

# Given:
str1 = "Emma25 is Data scientist50 and AI Expert"
# Expected Output:
# Emma25
# scientist50
```

```
# print("The original string is : " + str1)
res=[]
temp=str1.split(" ")

for item in temp:
  if any(char.isdigit() for char in item) and any(char.isalpha() for ch in item):
    res.append(item)

# print("Displaying words with alphabets and numbers")
# for i in res:
#     print(i)

# output:
# The original string is : Emma25 is Data scientist50 and AI Expert
# Displaying words with alphabets and numbers
# Emma25
# scientist50

# Exercise 18:
# Replace each special symbol with # in the following string
# Given:
str1 = '/*Jon is @developer & musician!!'
# Expected Output: ##Jon is #developer # musician##

import re
res=re.sub(r'[^\w\s]',"#",str1)
print("The strings after replacement : ", res)

# or
import string

# Replace punctuations with #
replace_char = '#'

# string.punctuation to get the list of all special symbols
for char in string.punctuation:
```

```
    str1 = str1.replace(char, replace_char)

print("The strings after replacement : ", str1)

# output:
# The strings after replacement :  ##Jon is #developer # musician##
```

 # 11) Authentication Ask username, password and compare with encryption (DESAES algo)

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad

# create key of 8 bytes
key=b'abcdefgh'

# create cipher object
cipher=DES.new(key,DES.MODE_ECB)

# print('cipher:',cipher)

# correct username and password
correct_username = "user123"
correct_password = "pass123"

encrypted_password=cipher.encrypt(pad(correct_password.encode(),8))
# print("encyrpted pass: ",encrypted_password)
# inputs
username = input("Enter username: ")
password = input("Enter password: ")

# pad is used to make data length mutliple of 8 bytes
encrypted_inputPass=cipher.encrypt(pad(password.encode(),8))
# print("input pass: ",encrypted_inputPass)

# compare
```

```
if username == correct_username and encrypted_inputPass == encrypted_password:
    print("Authentication successful!")
else:
    print("Authentication failed!")

# output
# Enter username: user123
# Enter password: pass123
# Authentication successful!
```

DARSHAN PRAJAPATI
MCA-1
ROLL NO: 24

PYTHON PRACTICAL ASSIGNMENT-2

LIST – Programs

1) Create a list & display elements

```
nums = [10, 20, 30, 40]
print("List:", nums)
```

output:

List: [10, 20, 30, 40]

2) Add element into a list

```
fruits = ["apple", "banana"]
fruits.append("orange")
print(fruits)
```

Output:

['apple', 'banana', 'orange']

3) Remove an element

```
colors = ["red", "blue", "green"]
colors.remove("blue")
print(colors)
```

Output:

['red', 'green']

4) Sort a list

```
marks = [45, 12, 90, 33]
marks.sort()
print(marks)
```

Output:

[12, 33, 45, 90]

5) Sum of all list elements

```
nums = [3, 5, 7, 9]
print("Sum =", sum(nums))
```

Outptu:

Sum = 24

6) Insert at index

```
a = [1, 2, 4]
a.insert(2, 3)
print(a)
```

Output:

[1, 2, 3, 4]

7) Pop last element

```
a = [5,6,7,8]
a.pop()
print(a)
```

Output:

[5, 6, 7]

8) Reverse list

```
a = [1,2,3,4,5]
a.reverse()
print(a)
```

Output:

[5, 4, 3, 2, 1]

9) Find max/min

```
a = [11,3,9,25]
print(max(a), min(a))
```

Output:

25 3

10) Loop through list

```
a = ["ram","shyam","rita"]
for i in a:
    print(i)
```

Output:

ram

shyam

rita

TUPLE – Programs

1) Create & display

```
t = (1, 2, 3, 4)
print(t)
```

Output:

(1, 2, 3, 4)

2) Access elements

```
t = ("A", "B", "C", "D")
print(t[2])
```

Output:

C

3) Count occurrences

```
t = (2, 4, 2, 2, 6)
print(t.count(2))
```

Output:

3

4) Convert tuple → list

```
t = (10, 20, 30)
lst = list(t)
print(lst)
```

Output:

[10, 20, 30]

5) Find max & min

```
t = (8, 3, 9, 1)
print(max(t), min(t))
```

Output:

9 1

6) Slice

```
t = (10,20,30,40,50)
print(t[1:4])
```

Output:

(20, 30, 40)

8) Length

```
t = (5,6,7,8)
print(len(t))
```

Output:

4

9) Check if element exists

```
t = (1,2,3,4)
print(2 in t)
```

Output:

True

10) Find index of value

```
t = (10,20,30,20)
print(t.index(20))
```

Output:

1

SET – Programs

1) Create a set

```
s = {1, 2, 3}
print(s)
```

Output:

{1, 2, 3}

2) Add element

```
s = {10, 20}
s.add(30)
print(s)
```

Output:

{10, 20, 30}

3) Remove element

```
s = {10, 20, 30}
s.remove(20)
print(s)
```

Output:

{10, 30}

4) Union of sets

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a.union(b))
```

Output:

{1, 2, 3, 4, 5}

5) Intersection

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a.intersection(b))
```

Output:

{2, 3}

6) Discard item

```
s = {5,6,7}
s.discard(6)
print(s)
```

Output:

{5, 7}

7) Clear set

```
s = {1,2,3}
s.clear()
print(s)
```

Output:

set()

8) Difference

```
a = {1,2,3,4}
b = {3,4}
print(a.difference(b))
```

Output:

{1, 2}

9) Symmetric difference

```
a = {1,2,3}
b = {3,4,5}
print(a.symmetric_difference(b))
```

Output:

{1, 2, 4, 5}

10) Loop through set

```
s = {"a","b","c"}
for x in s:
    print(x)
```

Output:

c

a

b

<u>DICTIONARY – Programs</u>

1) Create & display

```
student = {"name": "Darshan", "age": 21}
print(student)
```

Output:

{'name': 'Darshan', 'age': 21}

2) Add key-value

```
student = {"name": "DP"}
student["course"] = "MCA"
print(student)
```

Output:

{'name': 'DP', 'course': 'MCA'}

3) Remove a key

```
data = {"a":1, "b":2, "c":3}
data.pop("b")
print(data)
```

Output:

{'a': 1, 'c': 3}

4) Loop through dictionary

```
d = {"x":10, "y":20, "z":30}
for k, v in d.items():
    print(k, "=", v)
```

Output:

x = 10

y = 20

z = 30

5) Get only keys / only values

```
d = {"id":101, "name":"Ram", "marks":85}
print(d.keys())
print(d.values())
```

Output:

dict_keys(['id', 'name', 'marks'])

dict_values([101, 'Ram', 85])

STRING – Programs

1) Create a string & print it

```
name = "Darshan Prajapati"
print(name)
```

Output:

Darshan Prajapati

2) Length of string

```
text = "Hello World"
print("Length =", len(text))
```

Output:

Length = 11

3) Count a specific character

```
msg = "banana"
print(msg.count('a'))
```

Output:

3

4) Reverse a string

```
s = "python"
rev = s[::-1]
print(rev)
```

Output:

nohtyp

5) Check if string is palindrome

```
s = "madam"
if s == s[::-1]:
```

```
    print("Palindrome")
else:
    print("Not Palindrome")
```

Output:

Palindrome